VECTONIX TECHNOLOGIES

Winter Internship 2026

FULL STACK DEVELOPMENT LEARNING HANDBOOK

From Zero to Full Stack Developer in 30 Days

HTML • CSS • JavaScript • React • Next.js • Node.js • SQL • Git

vectonix.in | 25 November – 25 December 2026

Table of Contents

Ch.	Topic	
1	Introduction to Full Stack Development	
2	HTML — Building Web Pages	
3	CSS — Styling Your Web Pages	
4	JavaScript — Making Pages Interactive	
5	React.js — Component-Based Development	
6	Next.js Basics	
7	Backend Development with Node.js & Express	
8	Database and SQL	
9	Git and GitHub — Version Control	
10	Deployment — Taking Your App Live	
P	Practical Projects	
R	Learning Resources	

CHAPTER 1

Introduction to Full Stack Development

Understanding how modern web applications work

1.1 What is Web Development?

Web development is the process of building websites and web applications that people can access through a browser (like Chrome or Firefox). Every website you visit — from Google to YouTube to Instagram — was built by web developers.

Think of a website like a restaurant. The **front of the restaurant** — the menu, décor, tables, and waiters — is what customers see and interact with. The **kitchen behind the scenes** prepares the food, manages orders, and stores ingredients. Web development works the same way.

1.2 Frontend, Backend, and Database

Part	What It Is	What It Does	Technologies Used
Frontend	The user interface	Shows buttons, text, images — what you see	HTML, CSS, JavaScript, React
Backend	The server logic	Processes requests, applies business rules	Node.js, Python, Java
Database	Data storage	Stores and retrieves data permanently	MySQL, PostgreSQL, MongoDB

1.3 How a Website Works — Client-Server Architecture

When you type a URL in your browser and press Enter, here is exactly what happens:

Step	What Happens	Who Does It
1. Request	Browser sends a request to the server	Client (your browser)
2. DNS	Domain name is converted to an IP address	DNS Server
3. Server	Server receives the request, runs logic	Backend (Node.js/Python)
4. Database	Server fetches data from the database	Database (SQL/NoSQL)
5. Response	Server sends HTML/CSS/JS back to the browser	Backend → Client
6. Render	Browser displays the page to the user	Client (browser engine)

1.4 Real-World Examples

Here are technologies behind apps you use every day:

Application	Frontend	Backend	Database

Facebook/Meta	React.js	PHP, Python	MySQL, Cassandra
YouTube	React/Angular	Java, Python	Bigtable, Spanner
Netflix	React.js	Node.js, Java	Cassandra, MySQL
Amazon	React.js	Java, Python	DynamoDB, Aurora
Swiggy/Zomato	React.js	Node.js, Go	PostgreSQL, Redis

Chapter 1 — Exercises & Self-Assessment

Part A: Multiple Choice Questions

1. What does 'frontend' mean in web development?

- A) The server that processes data
- B) The user interface that people see
- C) The database storage
- D) The hosting platform

2. Which of these is a backend technology?

- A) HTML
- B) CSS
- C) Node.js
- D) React.js

3. What is the role of a database?

- A) Show images on screen
- B) Style the page
- C) Store and retrieve data
- D) Handle browser events

4. In client-server architecture, who is the 'client'?

- A) The developer
- B) The database
- C) The user's browser
- D) The web server

5. Which company uses React.js for its frontend?

- A) Facebook
- B) Netflix
- C) Amazon
- D) All of the above

Part B: Short Answer Questions

1. In your own words, explain the difference between frontend and backend.

2. Name three real-world web applications and describe what they do.

3. What happens when you press Enter after typing a URL in a browser?

4. Why do we need a separate database instead of storing data in code?

5. What is the difference between a website and a web application?

Part C: Practical Exercises

Exercise 1: Draw a diagram (on paper or digitally) showing how a client, server, and database interact when a user logs into a website.

Exercise 2: List 5 websites you use daily. For each one, guess which frontend framework and backend language they might use.

Part D: Real-World Scenario

You are building an online bookstore. A customer searches for 'Python programming'. Describe, step by step, what happens from the moment they type the search to the moment results appear on screen. Mention the role of frontend, backend, and database in your answer.

Answer Key

• MCQ: 1-B, 2-C, 3-C, 4-C, 5-D

• Short 1: Frontend is what users see (HTML/CSS/JS); backend processes data on the server.

• Short 2: Examples: Gmail (email), Zomato (food delivery), Amazon (shopping).

• Short 3: Browser sends HTTP request → DNS resolves domain → server processes → returns HTML/CSS/JS → browser renders.

• Short 4: Data in code is lost when the program stops; databases persist data permanently.

• Short 5: Website is mostly static content; web application is interactive and dynamic.

CHAPTER 2

HTML

HyperText Markup Language — The Skeleton of Every Web Page

2.1 What is HTML?

HTML stands for **HyperText Markup Language**. Let's break that down:

- HyperText — Text that contains links to other documents (hyperlinks).
- Markup — Tags that describe the structure and meaning of content.
- Language — A set of rules that browsers understand.

HTML is the **skeleton** of every web page. It tells the browser what content to show: headings, paragraphs, images, links, forms. It does NOT control how things look (that's CSS) or how they behave (that's JavaScript).

2.2 HTML Document Structure

Every HTML page follows this structure:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>My First Web Page</title>
</head>
<body>
<h1>Hello, World!</h1>
<p>This is my first web page.</p>
</body>
</html>
```

Line-by-line explanation:

Line	Code	What It Does
1	<!DOCTYPE html>	Tells browser this is HTML5 (modern HTML)
2	<html lang="en">	Root element; lang="en" sets language to English
3	<head>	Contains invisible page settings (not shown on page)
4	<meta charset="UTF-8">	Supports all characters including special symbols
5	<meta name="viewport">	Makes page work on mobile screens

6	<title>...</title>	Shows text in browser tab
7	</head>	Closes the head section
8	<body>	Everything visible on the page goes here
9	...	Largest heading (most important title)
10	...	A paragraph of text
11	</body>	Closes the body section
12	</html>	Closes the HTML document

2.3 Text Elements: Headings and Paragraphs

```
<!-- Headings go from h1 (largest) to h6 (smallest) -->
<h1>Main Title of the Page</h1>
<h2>Section Heading</h2>
<h3>Sub-section Heading</h3>
<h4>Smaller Heading</h4>

<!-- Paragraphs -->
<p>This is a paragraph. It wraps text automatically.</p>

<!-- Line break (no closing tag) -->
<p>Line one<br>Line two</p>

<!-- Bold and italic -->
<p>This word is <strong>bold</strong> and this is <em>italic</em>.</p>
```

2.4 Links

```
<!-- Basic link -->
<a href="https://vectonix.in">Visit Vectonix</a>

<!-- Open in new tab -->
<a href="https://vectonix.in" target="_blank">Open in new tab</a>

<!-- Link to another page in same project -->
<a href="about.html">About Us</a>

<!-- Link to a section on the same page -->
<a href="#contact">Go to Contact</a>
<section id="contact">Contact info here</section>
```

2.5 Images

```
<!-- Basic image -->
<img src="photo.jpg" alt="A description of the image">
```

```
<!-- Image with size -->
<img src="logo.png" alt="Company Logo" width="200" height="100">

<!-- Image from a URL -->
<img src="https://vectonix.in/logo.png" alt="Vectonix Logo">
```

The **alt** attribute is required for accessibility — it describes the image for screen readers and shows if the image fails to load.

2.6 Lists

```
<!-- Unordered list (bullet points) -->
<ul>
<li>HTML</li>
<li>CSS</li>
<li>JavaScript</li>
</ul>

<!-- Ordered list (numbered) -->
<ol>
<li>Sign up</li>
<li>Verify email</li>
<li>Log in</li>
</ol>
```

2.7 Tables

```
<table>
<thead>
<tr>
<th>Name</th>
<th>Course</th>
<th>Grade</th>
</tr>
</thead>
<tbody>
<tr>
<td>Aisha Khan</td>
<td>Computer Science</td>
<td>A</td>
</tr>
<tr>
```

```
<td>Rahul Sharma</td>
<td>Information Technology</td>
<td>B+</td>
</tr>
</tbody>
</table>
```

2.8 Forms

Forms collect information from users — like login pages or registration forms.

```
<form action="/submit" method="POST">

<!-- Text input -->
<label for="name">Full Name:</label>
<input type="text" id="name" name="name" placeholder="Enter your name" required>

<!-- Email input -->
<label for="email">Email:</label>
<input type="email" id="email" name="email" required>

<!-- Password -->
<label for="pass">Password:</label>
<input type="password" id="pass" name="password" required>

<!-- Radio buttons -->
<p>Gender:</p>
<input type="radio" name="gender" value="male"> Male
<input type="radio" name="gender" value="female"> Female

<!-- Checkbox -->
<input type="checkbox" name="agree" required>
<label>I agree to terms</label>

<!-- Dropdown -->
<select name="course">
<option value="fsd">Full Stack Development</option>
<option value="cyber">Cybersecurity</option>
<option value="dm">Digital Marketing</option>
</select>

<!-- Submit button -->
<button type="submit">Register Now</button>

</form>
```

2.9 Semantic HTML

Semantic HTML uses tags that **describe their meaning**, not just their appearance. This helps search engines understand your page and makes code easier to read.

Tag	Meaning	When to Use
<header>	Top section of page	Logo, navigation, site title
<nav>	Navigation links	Menu, sidebar links
<main>	Main content	The primary content area
<article>	Self-contained content	Blog posts, news articles
<section>	Grouped content	Chapters, tabs, topic groups
<aside>	Side content	Sidebar, ads, related links
<footer>	Bottom section	Copyright, contact, links

2.10 Practical Project: Student Registration Page

Let's build a complete student registration page using everything we've learned:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Student Registration - Vectonix Internship</title>
</head>
<body>
<header>
<h1>Vectonix Technologies</h1>
<nav>
<a href="index.html">Home</a> |
<a href="register.html">Register</a> |
<a href="contact.html">Contact</a>
</nav>
</header>

<main>
<h2>Winter Internship 2026 — Registration</h2>
<p>Fill in your details below to apply.</p>

<form action="/register" method="POST">
<label for="fname">First Name:</label>
<input type="text" id="fname" name="firstName" required>
```

```
<label for="lname">Last Name:</label>
<input type="text" id="lname" name="lastName" required>

<label for="email">Email Address:</label>
<input type="email" id="email" name="email" required>

<label for="college">College / University:</label>
<input type="text" id="college" name="college" required>

<label for="domain">Choose Your Domain:</label>
<select id="domain" name="domain">
<option value="">-- Select --</option>
<option value="fsd">Full Stack Development</option>
<option value="cyber">Cybersecurity</option>
<option value="dm">Digital Marketing</option>
</select>

<input type="checkbox" name="agree" required>
<label>I agree to the internship terms and conditions</label>

<button type="submit">Submit Application</button>
</form>
</main>

<footer>
<p>&copy; 2026 Vectonix Technologies | vectonix.in</p>
</footer>
</body>
</html>
```

Chapter 2 — Exercises & Self-Assessment

Part A: Multiple Choice Questions

1. What does the <head> tag contain?

- A) Visible page content
- B) Page metadata and settings
- C) Navigation links
- D) Form elements

2. Which tag creates a hyperlink?

- A) <link>
- B) <url>
- C) <a>
- D) <href>

3. What attribute makes a form field required?

- A) mandatory
- B) must-fill
- C) required
- D) validate

4. Which tag is used for the largest heading?

- A) <h6>
- B) <header>
- C) <h1>
- D) <title>

5. What is 'semantic HTML'?

- A) HTML with animations
- B) HTML tags that describe their meaning
- C) HTML without CSS
- D) Advanced HTML5

Part B: Short Answer Questions

1. What is the difference between and <b> in HTML?

2. Why is the alt attribute important for images?

3. What is the difference between <ul> and ?

4. Explain the difference between action and method attributes in a <form>.

5. Name four semantic HTML5 tags and explain when you would use each.

Part C: Practical Exercises

Exercise 1: Create an HTML page with: a heading, a paragraph about yourself, an image, a list of your hobbies, and a link to your college website.

Exercise 2: Build a contact form with fields: name, email, subject, message, and a submit button. Make all fields required.

Part D: Real-World Scenario

You are creating a website for a local restaurant. The restaurant needs: a homepage with the restaurant name and a photo, a menu page with a table of dishes and prices, and a reservations page with a booking form. Write the HTML for the homepage.

Answer Key

• MCQ: 1-B, 2-C, 3-C, 4-C, 5-B

• Short 1: has semantic meaning (important), <b> is purely visual (bold).

• Short 2: Accessibility for screen readers; displays when image fails to load; SEO.

• Short 3: <ul> is unordered (bullet points); is ordered (numbered list).

• Short 4: action = URL where form data is sent; method = HTTP method (GET/POST).

• Short 5: <header> (top of page), <nav> (navigation), <main> (main content), <footer> (bottom).

CHAPTER 3

CSS

Cascading Style Sheets — Making Your Pages Beautiful

3.1 What is CSS?

CSS stands for **Cascading Style Sheets**. It controls how HTML elements look — their colours, fonts, sizes, spacing, and layout. Without CSS, every web page would be plain black text on a white background.

3.2 Three Ways to Add CSS

Method	How	Example	Best For
Inline	style attribute on element	Text	Quick single-element overrides
Internal	<style> tag in <head>	<style> p { color: red; } </style>	Single-page styles
External	Separate .css file	<link rel="stylesheet" href="style.css">	Full projects (recommended)

3.3 Selectors

```
/* Element selector — targets all <p> tags */
p { color: #1A1A2E; }

/* Class selector — targets elements with class="highlight" */
.highlight { background-color: yellow; }

/* ID selector — targets element with id="header" */
#header { font-size: 24px; }

/* Universal selector — targets everything */
* { box-sizing: border-box; margin: 0; padding: 0; }

/* Descendant — <a> tags inside <nav> only */
nav a { color: white; }
```

3.4 Colours, Fonts, and Backgrounds

```
/* Colour formats */
color: red; /* named colour */
color: #6C3BCE; /* hex colour */
color: rgb(108, 59, 206); /* rgb */
color: rgba(108,59,206,0.8); /* rgba (with transparency) */
```

```
/* Fonts */
font-family: 'Segoe UI', Arial, sans-serif;
font-size: 16px;
font-weight: bold; /* or 400, 700 */
font-style: italic;
line-height: 1.6;

/* Backgrounds */
background-color: #EDE7F6;
background-image: url('bg.jpg');
background: linear-gradient(135deg, #6C3BCE, #9C6FE0); /* gradient */
```

3.5 The Box Model

Every HTML element is a box. The box model describes the layers around content:

Layer	What It Is	Example
Content	The actual text or image	width: 200px; height: 100px;
Padding	Space inside the border	padding: 20px;
Border	Line around the element	border: 2px solid #6C3BCE;
Margin	Space outside the border	margin: 10px auto;

```
.card {
width: 300px;
padding: 20px; /* space inside */
border: 2px solid #6C3BCE;
margin: 10px; /* space outside */
border-radius: 8px; /* rounded corners */
box-shadow: 0 2px 8px rgba(0,0,0,0.1); /* shadow */
}
```

3.6 Flexbox

Flexbox makes it easy to align and distribute items in a container. It's perfect for navigation bars, card layouts, and centering content.

```
.navbar {
display: flex;
justify-content: space-between; /* spread items left and right */
align-items: center; /* vertical center */
padding: 10px 20px;
background-color: #6C3BCE;
```

```
}

.cards-container {
display: flex;
flex-wrap: wrap; /* wrap to next line if needed */
gap: 20px; /* space between cards */
justify-content: center;
}
```

3.7 Responsive Design and Media Queries

Responsive design means your page looks good on all screen sizes — phones, tablets, and desktops. Media queries apply different styles based on screen width.

```
/* Default styles (for desktop) */
.container { width: 1200px; margin: 0 auto; }

/* Tablet (max 768px wide) */
@media (max-width: 768px) {
.container { width: 100%; padding: 0 20px; }
.cards-container { flex-direction: column; }
}

/* Mobile (max 480px wide) */
@media (max-width: 480px) {
.navbar { flex-direction: column; gap: 10px; }
h1 { font-size: 22px; }
}
```

Chapter 3 — Exercises & Self-Assessment

Part A: Multiple Choice Questions

1. Which CSS property adds space INSIDE an element's border?

- A) margin
- B) padding
- C) border
- D) gap

2. Which selector targets an element with id='header'?

- A) .header
- B) *header
- C) #header
- D) header

3. Which display value enables flexbox?

- A) display: block
- B) display: grid
- C) display: inline
- D) display: flex

4. What does 'responsive design' mean?

- A) Fast loading website
- B) Website that works on all screen sizes
- C) Animated website
- D) SEO-friendly website

5. Which CSS rule applies only when screen width is less than 768px?

- A) @screen (max: 768px)
- B) @media max-width: 768px
- C) @media (max-width: 768px)
- D) #media (width < 768px)

Part B: Short Answer Questions

1. Explain the difference between margin and padding with an example.

2. What is the box model? Name all four layers.

3. What is the advantage of using an external CSS file over inline styles?

4. Write CSS to centre a div horizontally using flexbox.

5. What is a CSS class selector? How is it different from an ID selector?

Part C: Practical Exercises

Exercise 1: Style your Chapter 2 registration page: add a purple background to the header, set font to Arial, add padding to all inputs, and make the submit button purple with white text.

Exercise 2: Create a responsive card grid: 3 cards side by side on desktop, 2 on tablet (768px), 1 on mobile (480px).

Part D: Real-World Scenario

A startup wants their landing page to look professional. Describe how you would use CSS to: style the navigation bar, create a hero section with a gradient background, display features in a 3-column grid, and make the page responsive for mobile.

Answer Key

• MCQ: 1-B, 2-C, 3-D, 4-B, 5-C

• Short 1: padding is inside border; margin is outside. E.g., padding:20px adds space inside the box.

• Short 2: Content, Padding, Border, Margin — four layers around every HTML element.

• Short 3: External CSS is reusable across pages; easier to maintain; keeps HTML clean.

• Short 4: .parent { display: flex; justify-content: center; } or margin: 0 auto; with width set.

• Short 5: Class (.) can apply to multiple elements; ID (#) must be unique on a page.

CHAPTER 4

JavaScript

Making Web Pages Interactive and Dynamic

4.1 What is JavaScript?

JavaScript (JS) is the programming language of the web. While HTML creates structure and CSS adds style, **JavaScript makes pages come alive**. Every time you click a button and something happens without reloading the page — that's JavaScript.

4.2 Variables: var, let, const

```
// var – old way, function-scoped (avoid in modern JS)
var name = 'Rahul';

// let – modern, block-scoped, value can change
let age = 21;
age = 22; // OK

// const – block-scoped, value cannot be reassigned
const PI = 3.14159;
// PI = 3; // Error!

// Rule: Use const by default, let when you need to change value
```

4.3 Data Types

Type	Example	Description
String	"Hello World"	Text — always in quotes
Number	42, 3.14	Integers and decimals
Boolean	true, false	Only two values: true or false
Array	[1, 2, 3]	Ordered list of values
Object	{name: 'Ali'}	Key-value pairs
null	null	Intentionally empty value
undefined	undefined	Variable declared but not assigned

4.4 Conditionals and Loops

```
// If-else
const score = 75;
if (score >= 90) {
console.log('Grade: A');
```

```
} else if (score >= 75) {
console.log('Grade: B'); // This runs
} else {
console.log('Grade: C');
}

// For loop
for (let i = 1; i <= 5; i++) {
console.log('Iteration: ' + i);
}
// Output: Iteration: 1, 2, 3, 4, 5

// Array with forEach
const fruits = ['Apple', 'Mango', 'Banana'];
fruits.forEach(fruit => {
console.log(fruit);
});
```

4.5 Functions

```
// Function declaration
function greet(name) {
return 'Hello, ' + name + '!';
}
console.log(greet('Priya')); // Hello, Priya!

// Arrow function (modern, shorter syntax)
const add = (a, b) => a + b;
console.log(add(5, 3)); // 8

// Default parameters
const welcome = (name = 'Student') => `Welcome, ${name}!`;
console.log(welcome()); // Welcome, Student!
console.log(welcome('Arjun')); // Welcome, Arjun!
```

4.6 DOM Manipulation

DOM = Document Object Model. JavaScript can find and change HTML elements on the page.

```
// HTML: <h1 id='title'>Old Title</h1>
// <p class='intro'>Old text</p>

// Find element by ID
const title = document.getElementById('title');
title.textContent = 'New Title!';
```

```
// Find by CSS selector
const intro = document.querySelector('.intro');
intro.textContent = 'Updated text!';
intro.style.color = '#6C3BCE';

// Create a new element
const newPara = document.createElement('p');
newPara.textContent = 'I was added by JavaScript!';
document.body.appendChild(newPara);
```

4.7 Events

```
// HTML: <button id='myBtn'>Click Me</button>

const btn = document.getElementById('myBtn');

btn.addEventListener('click', function() {
alert('Button clicked!');
});

// Common events: click, submit, keyup, mouseover, change

// Form submission
const form = document.querySelector('form');
form.addEventListener('submit', function(event) {
event.preventDefault(); // stop page reload
console.log('Form submitted!');
});
```

4.8 Fetch API — Getting Data from a Server

```
// Fetch data from an API
fetch('https://api.example.com/users')
.then(response => response.json()) // parse JSON
.then(data => {
console.log(data); // array of users
data.forEach(user => {
console.log(user.name);
});
})
.catch(error => {
console.error('Error:', error);
});

// Modern async/await version (cleaner)
```

```
async function getUsers() {
try {
const response = await fetch('https://api.example.com/users');
const data = await response.json();
return data;
} catch (error) {
console.error('Failed:', error);
}
}
```

Chapter 4 — Exercises & Self-Assessment

Part A: Multiple Choice Questions

1. What keyword creates a variable that cannot be reassigned?

- A) var
- B) let
- C) const
- D) static

2. What does DOM stand for?

- A) Data Object Model
- B) Document Object Model
- C) Dynamic Output Manager
- D) Design Object Method

3. Which method adds an event listener to an element?

- A) element.on('click', fn)
- B) element.listen('click', fn)
- C) element.addEventListener('click', fn)
- D) element.click(fn)

4. What does fetch() return?

- A) A string
- B) An array
- C) A Promise
- D) A number

5. Which arrow function syntax is correct?

- A) fn = (x) -> x*2
- B) fn = (x) => x*2
- C) fn: (x) => x*2
- D) function (x) => x*2

Part B: Short Answer Questions

1. What is the difference between let and const?

2. What does event.preventDefault() do and when would you use it?

3. Explain what 'asynchronous JavaScript' means in simple terms.

4. What is the difference between textContent and innerHTML?

5. Write a function that takes an array of numbers and returns their sum.

Part C: Practical Exercises

Exercise 1: Build a calculator: HTML with two input fields and buttons for +, -, *, /. JavaScript reads values and displays result without page reload.

Exercise 2: Create a to-do list: an input + Add button. On click, add the task to a <ul>. Add a Delete button next to each task.

Part D: Real-World Scenario

A form has fields for name, email, and password. Write JavaScript that: validates all fields are filled, checks email contains '@', checks password is at least 8 characters, shows an error message for each failed check, and only submits if all pass.

Answer Key

• MCQ: 1-C, 2-B, 3-C, 4-C, 5-B

• Short 1: let can be reassigned; const cannot be reassigned after declaration.

• Short 2: Prevents default browser behaviour (e.g., stops form from reloading page on submit).

• Short 3: Code continues running while waiting for slow operations (network, file reads) to complete.

• Short 4: textContent sets plain text (safe); innerHTML sets HTML markup (can cause XSS if not careful).

• Short 5: const sum = arr => arr.reduce((total, n) => total + n, 0);

CHAPTER 5

React.js

Building Modern User Interfaces with Components

5.1 What is React?

React is a JavaScript library created by Facebook (now Meta) for building user interfaces. Instead of writing HTML directly, React lets you build **reusable components** — like building with LEGO blocks. Each component is a self-contained piece of UI.

Traditional HTML/JS	React
Update DOM directly with getElementById	State changes automatically update the UI
Code gets messy for large apps	Components keep code organised
Hard to reuse code	Components are reusable everywhere
No automatic re-renders	React re-renders only what changed

5.2 Creating a React App

```
# Create with Vite (faster, recommended)
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev

# OR with create-react-app
npx create-react-app my-app
cd my-app
npm start
```

5.3 Components and JSX

A component is a JavaScript function that returns JSX (HTML-like syntax inside JavaScript).

```
// A simple functional component
function WelcomeCard() {
return (
<div className='card'>
<h2>Welcome to Vectonix!</h2>
<p>Your internship starts now.</p>
</div>
);
```

```
}

// Use it like an HTML tag
function App() {
return (
<div>
<WelcomeCard />
<WelcomeCard /> {/* Reuse it! */}
</div>
);
}
```

5.4 Props — Passing Data to Components

```
// Component that accepts props
function StudentCard({ name, domain, college }) {
return (
<div className='student-card'>
<h3>{name}</h3>
<p>Domain: {domain}</p>
<p>College: {college}</p>
</div>
);
}

// Using the component with different data
function App() {
return (
<div>
<StudentCard name='Priya Sharma' domain='FSD' college='VIT' />
<StudentCard name='Arjun Nair' domain='Cybersecurity' college='BITS' />
</div>
);
}
```

5.5 State — Dynamic Data with useState

```
import { useState } from 'react';

function Counter() {
// useState returns [currentValue, setterFunction]
const [count, setCount] = useState(0);

return (
```

```
<div>
<h2>Count: {count}</h2>
<button onClick={() => setCount(count + 1)}>+1</button>
<button onClick={() => setCount(count - 1)}>-1</button>
<button onClick={() => setCount(0)}>Reset</button>
</div>
);
}
```

5.6 useEffect — Running Code on Events

```
import { useState, useEffect } from 'react';

function UserList() {
const [users, setUsers] = useState([]);

// Runs once when component mounts (loads)
useEffect(() => {
fetch('https://jsonplaceholder.typicode.com/users')
.then(res => res.json())
.then(data => setUsers(data));
}, []); // [] = run only once

return (
<ul>
{users.map(user => (
<li key={user.id}>{user.name}</li>
))}
</ul>
);
}
```

5.7 Practical: Task Management App

```
import { useState } from 'react';

function TaskApp() {
const [tasks, setTasks] = useState([]);
const [input, setInput] = useState('');

const addTask = () => {
if (input.trim() === '') return;
setTasks([...tasks, { id: Date.now(), text: input, done: false }]);
setInput('');
```

```
};

const toggleTask = (id) => {
setTasks(tasks.map(t =>
t.id === id ? { ...t, done: !t.done } : t
));
};

const deleteTask = (id) => {
setTasks(tasks.filter(t => t.id !== id));
};

return (
<div>
<h1>My Tasks</h1>
<input value={input} onChange={e => setInput(e.target.value)}
placeholder='Enter a task...' />
<button onClick={addTask}>Add Task</button>
<ul>
{tasks.map(task => (
<li key={task.id}
style={{ textDecoration: task.done ? 'line-through' : 'none' }}>
{task.text}
<button onClick={() => toggleTask(task.id)}>✓</button>
<button onClick={() => deleteTask(task.id)}>X</button>
</li>
))}
</ul>
<p>{tasks.filter(t => t.done).length}/{tasks.length} completed</p>
</div>
);
}

export default TaskApp;
```

Chapter 5 — Exercises & Self-Assessment

Part A: Multiple Choice Questions

1. What is JSX?

- A) A database query language
- B) HTML-like syntax used in React
- C) A CSS framework
- D) A Node.js module

2. What hook manages local state in a React component?

- A) useContext
- B) useReducer
- C) useState
- D) useRef

3. How do you pass data from a parent to a child component?

- A) Using state
- B) Using global variables
- C) Using props
- D) Using useEffect

4. When does useEffect with an empty dependency array [] run?

- A) On every render
- B) Only once when component mounts
- C) Only on unmount
- D) Never

5. Why do list items in React need a key prop?

- A) For styling
- B) To help React track and update items efficiently
- C) Required by HTML
- D) For accessibility

Part B: Short Answer Questions

1. What is the difference between state and props in React?

2. Why are components reusable and why does that matter?

3. What happens when you call a state setter function (e.g., setCount)?

4. When would you use useEffect? Give two real-world examples.

5. Write a React component that displays a list of fruits from an array.

Part C: Practical Exercises

Exercise 1: Build a colour picker app: 3 buttons (Red, Green, Blue) that change the background colour of a using useState.

Exercise 2: Build a search filter: a list of 10 student names, an input field. As you type, filter the list to show only matching names.

Part D: Real-World Scenario

You are building a shopping cart for an e-commerce site. Describe which React concepts you would use for: (1) displaying product cards, (2) adding items to cart, (3) showing the cart count in the navbar, (4) fetching products from an API on page load.

Answer Key

• MCQ: 1-B, 2-C, 3-C, 4-B, 5-B

• Short 1: Props are read-only data passed from parent; state is local mutable data managed by the component itself.

• Short 2: A component written once can be used many times with different data, reducing code duplication.

• Short 3: React re-renders the component with the new state value.

• Short 4: 1) Fetch data from API when page loads; 2) Set up a timer or subscription.

• Short 5: function FruitList() { const fruits = ['Apple','Mango','Banana']; return {fruits.map((f,i) => <li key={i}>{f})}; }

CHAPTER 6

Next.js Basics

Server-Side React for Production-Ready Apps

6.1 What is Next.js?

Next.js is a framework built on top of React that adds server-side rendering (SSR), file-based routing, and built-in optimisations. Think of React as the engine and Next.js as the complete car with all features included.

Feature	React	Next.js
Routing	Need React Router	Built-in file-based routing
SEO	Poor (CSR)	Excellent (SSR/SSG)
API routes	Separate backend needed	Built-in /api routes
Image optimisation	Manual	Automatic with <Image>
Setup	More configuration	Zero config to start

6.2 Creating a Next.js Project

```
npx create-next-app@latest my-nextjs-app
cd my-nextjs-app
npm run dev
# Open http://localhost:3000
```

6.3 File-Based Routing

In Next.js App Router, every folder with a page.js file becomes a route:

```
app/
■■■ page.js → / (homepage)
■■■ about/
■ ■■■ page.js → /about
■■■ blog/
■ ■■■ page.js → /blog
■ ■■■ [id]/
■ ■■■ page.js → /blog/1, /blog/2 (dynamic route)
■■■ api/
■■■ users/
■■■ route.js → /api/users (API endpoint)
```

6.4 A Simple Page Component

```
// app/page.js — Homepage
export default function HomePage() {
return (
<main>
<h1>Welcome to Vectonix Internship</h1>
<p>Learn Full Stack Development in 30 days.</p>
</main>
);
}
```

6.5 API Routes

```
// app/api/hello/route.js
export async function GET() {
return Response.json({ message: 'Hello from API!', status: 'ok' });
}

// Test it: fetch('/api/hello') or visit http://localhost:3000/api/hello
```

Chapter 6 — Exercises & Self-Assessment

Part A: Multiple Choice Questions

1. What makes Next.js routing different from React Router?

- A) Next.js uses hash routing
- B) Next.js uses file-based routing
- C) Next.js has no routing
- D) Next.js requires XML configuration

2. What folder contains API routes in Next.js App Router?

- A) /pages/api
- B) /routes
- C) /app/api
- D) /server

3. What command starts a Next.js development server?

- A) npm start
- B) npm run build
- C) npm run dev
- D) npx next

4. How do you create a dynamic route for /blog/[id] in App Router?

- A) app/blog/id/page.js
- B) app/blog/[id]/page.js
- C) app/blog/:id/page.js
- D) app/blog/dynamic/page.js

5. What is the main advantage of Next.js SSR over React CSR for SEO?

- A) Faster JavaScript
- B) More components
- C) HTML is pre-rendered so search engines can read it
- D) Better CSS support

Part B: Short Answer Questions

1. What is the difference between SSR (server-side rendering) and CSR (client-side rendering)?

2. Why would you use Next.js instead of plain React for a commercial project?

3. What is an API route in Next.js and what can it do?

4. Explain file-based routing with a simple example.

5. What file do you create for the homepage in the Next.js App Router?

Part C: Practical Exercises

Exercise 1: Create a Next.js app with three pages: Home, About, and Contact. Add navigation links between them.

Exercise 2: Create an API route at /api/tasks that returns a JSON array of 3 tasks. Fetch and display them on the homepage.

Part D: Real-World Scenario

Your client wants a blog website where pages load fast and Google can find their posts. Explain why you would choose Next.js and describe how you would structure the project with pages for the homepage, a blog list, and individual blog posts.

Answer Key

• MCQ: 1-B, 2-C, 3-C, 4-B, 5-C

• Short 1: SSR renders HTML on the server for each request; CSR renders in the browser using JavaScript.

• Short 2: SEO, file-based routing, API routes, image optimisation, and better performance.

• Short 3: A serverless function inside Next.js that handles HTTP requests; can query a database or return data.

• Short 4: app/about/page.js → accessible at /about URL automatically.

• Short 5: app/page.js

CHAPTER 7

Backend Development

Building APIs with Node.js and Express.js

7.1 What is Backend Development?

The backend is everything that happens on the server. It handles business logic, authentication, database operations, and returns data to the frontend. When you log in to an app, the backend checks your credentials and sends back a token.

7.2 What is an API?

An API (Application Programming Interface) is a set of rules for how software communicates. Think of it as a waiter in a restaurant: you (the frontend) place an order; the waiter (API) takes it to the kitchen (server); the kitchen prepares it and sends it back.

7.3 HTTP Methods

Method	Action	Use Case	Example
GET	Read data	Fetch a list of students	GET /api/students
POST	Create data	Register a new student	POST /api/students
PUT	Replace data	Update entire student record	PUT /api/students/1
PATCH	Partial update	Update only the email	PATCH /api/students/1
DELETE	Delete data	Remove a student	DELETE /api/students/1

7.4 Creating a Server with Express.js

```
// Install: npm init -y && npm install express

const express = require('express');
const app = express();

// Middleware to parse JSON request bodies
app.use(express.json());

// Simple GET route
app.get('/', (req, res) => {
res.json({ message: 'Vectonix API is running!' });
});

// Start the server on port 5000
app.listen(5000, () => {
console.log('Server running on http://localhost:5000');
```

```
});
```

7.5 Complete CRUD API — Students

```
const express = require('express');
const app = express();
app.use(express.json());

let students = [
{ id: 1, name: 'Priya Sharma', domain: 'FSD' },
{ id: 2, name: 'Arjun Nair', domain: 'Cybersecurity' },
];

// GET all students
app.get('/api/students', (req, res) => {
res.json(students);
});

// GET one student by ID
app.get('/api/students/:id', (req, res) => {
const student = students.find(s => s.id === parseInt(req.params.id));
if (!student) return res.status(404).json({ error: 'Not found' });
res.json(student);
});

// POST – create new student
app.post('/api/students', (req, res) => {
const { name, domain } = req.body;
if (!name || !domain) {
return res.status(400).json({ error: 'Name and domain required' });
}
const newStudent = { id: students.length + 1, name, domain };
students.push(newStudent);
res.status(201).json(newStudent);
});

// PUT – update student
app.put('/api/students/:id', (req, res) => {
const index = students.findIndex(s => s.id === parseInt(req.params.id));
if (index === -1) return res.status(404).json({ error: 'Not found' });
students[index] = { id: parseInt(req.params.id), ...req.body };
res.json(students[index]);
});
```

```
// DELETE student
app.delete('/api/students/:id', (req, res) => {
students = students.filter(s => s.id !== parseInt(req.params.id));
res.json({ message: 'Deleted successfully' });
});

app.listen(5000, () => console.log('API running on port 5000'));
```

Chapter 7 — Exercises & Self-Assessment

Part A: Multiple Choice Questions

1. Which HTTP method is used to CREATE a new resource?

- A) GET
- B) PUT
- C) POST
- D) PATCH

2. What does REST stand for?

- A) Really Easy Server Technology
- B) Representational State Transfer
- C) Remote Execution Service Tool
- D) Rendered Express Server Transport

3. What status code means 'Resource not found'?

- A) 200
- B) 201
- C) 400
- D) 404

4. What does express.json() middleware do?

- A) Converts response to JSON
- B) Parses JSON from the request body
- C) Validates JSON schema
- D) Encrypts JSON data

5. What is req.params.id in an Express route?

- A) The request body
- B) A URL path parameter
- C) A query string value
- D) A request header

Part B: Short Answer Questions

1. What is the difference between PUT and PATCH?

2. What does it mean for an API to be 'RESTful'?

3. What HTTP status code should you return when successfully creating a resource?

4. Why do we need middleware like express.json()?

5. What is the difference between req.body, req.params, and req.query?

Part C: Practical Exercises

Exercise 1: Create an Express API for a book library with CRUD operations. A book has id, title, author, and year.

Exercise 2: Add error handling: if the request body is missing required fields, return status 400 with a descriptive error message.

Part D: Real-World Scenario

A mobile app needs to show a news feed. Design the API: what endpoints would you create, what HTTP methods would each use, what data would each return, and what status codes would you use for success and error cases?

Answer Key

• MCQ: 1-C, 2-B, 3-D, 4-B, 5-B

• Short 1: PUT replaces the entire resource; PATCH updates only specified fields.

• Short 2: Uses standard HTTP methods, stateless, resources identified by URLs, returns JSON.

• Short 3: 201 Created.

• Short 4: Express cannot read JSON from the request body without this parser middleware.

• Short 5: req.body = request JSON; req.params = URL path params (:id); req.query = URL query strings (?key=value).

CHAPTER 8

Database and SQL

Storing and Retrieving Data Permanently

8.1 What is a Database?

A database is an organised collection of data stored on a computer. Think of it as a digital filing cabinet where all application data lives — users, orders, messages, products.

Type	Description	Examples	Best For
SQL (Relational)	Data in tables with rows/columns	MySQL, PostgreSQL, SQLite	Structured data, relationships
NoSQL	Flexible schema, various formats	MongoDB, Firebase, Redis	Unstructured or changing data

8.2 SQL Essentials

```
-- Create a table
CREATE TABLE students (
id SERIAL PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(150) UNIQUE NOT NULL,
domain VARCHAR(50),
enrolled_date DATE DEFAULT CURRENT_DATE
);

-- Insert data
INSERT INTO students (name, email, domain)
VALUES ('Priya Sharma', 'priya@email.com', 'FSD');

-- Read all students
SELECT * FROM students;

-- Read with condition
SELECT name, email FROM students WHERE domain = 'FSD';

-- Sort results
SELECT * FROM students ORDER BY name ASC;

-- Update a record
UPDATE students SET domain = 'Cybersecurity' WHERE id = 1;

-- Delete a record
DELETE FROM students WHERE id = 5;
```

```
-- Count records
SELECT domain, COUNT(*) AS total FROM students GROUP BY domain;
```

8.3 Joins — Combining Tables

Joins let you combine data from two or more related tables.

```
-- Two tables: students and projects
-- students: id, name, domain
-- projects: id, title, student_id (foreign key)

-- INNER JOIN: only rows that match in both tables
SELECT students.name, projects.title
FROM students
INNER JOIN projects ON students.id = projects.student_id;

-- LEFT JOIN: all students, even those without projects
SELECT students.name, projects.title
FROM students
LEFT JOIN projects ON students.id = projects.student_id;
```

Chapter 8 — Exercises & Self-Assessment

Part A: Multiple Choice Questions

1. What does PRIMARY KEY mean?

- A) The most important column
- B) A unique identifier for each row
- C) The first column
- D) A foreign table reference

2. Which SQL command retrieves data?

- A) GET
- B) FETCH
- C) SELECT
- D) READ

3. What does INNER JOIN return?

- A) All rows from left table
- B) All rows from both tables
- C) Only rows with matches in both tables
- D) Rows with no matches

4. Which clause filters rows in a SELECT statement?

- A) FILTER
- B) WHERE
- C) HAVING
- D) LIMIT

5. What is a FOREIGN KEY?

- A) An external database
- B) A column referencing PRIMARY KEY of another table
- C) An encrypted key
- D) A computed column

Part B: Short Answer Questions

1. What is the difference between SQL and NoSQL databases?

2. What is the difference between WHERE and HAVING in SQL?

3. Explain what a JOIN does with a real-world example.

4. Why would you use a database instead of storing data in a JavaScript array?

5. Write a SQL query to find all students enrolled in FSD domain, sorted by name.

Part C: Practical Exercises

Exercise 1: Create a database schema for a simple blog: tables for users, posts, and comments. Define columns, data types, and relationships.

Exercise 2: Write 5 SQL queries for the students table: count by domain, find students enrolled after a date, update email, delete by ID, join with a projects table.

Part D: Real-World Scenario

You are building a school management system. Identify at least 5 tables you would need, list the key columns for each, and describe the relationships between them using primary and foreign keys.

Answer Key

• MCQ: 1-B, 2-C, 3-C, 4-B, 5-B

• Short 1: SQL uses fixed tables/schemas; NoSQL uses flexible formats like documents or key-value.

• Short 2: WHERE filters individual rows before grouping; HAVING filters groups after GROUP BY.

• Short 3: INNER JOIN combines student and project tables by matching student_id, showing which student built which project.

• Short 4: Arrays are lost when server restarts; databases persist data, support queries, and scale.

• Short 5: SELECT * FROM students WHERE domain = 'FSD' ORDER BY name ASC;

CHAPTER 9

Git and GitHub

Version Control — Track, Collaborate, and Ship Code

9.1 What is Version Control?

Imagine you're writing an essay. You save a copy as 'essay_v1', then 'essay_v2', then 'essay_final', then 'essay_final_REAL'. Version control does this automatically and much better. **Git** tracks every change you make to code, so you can undo mistakes, collaborate with others, and understand the history of your project.

9.2 Essential Git Commands

Command	What It Does	Example
git init	Start a new Git repository	git init
git status	Show changed files	git status
git add <file>	Stage changes for commit	git add index.html
git add .	Stage ALL changes	git add .
git commit -m	Save staged changes with a message	git commit -m "Add login page"
git log	Show commit history	git log --oneline
git push origin main	Upload to GitHub	git push origin main
git pull	Download latest from GitHub	git pull
git clone <url>	Copy a GitHub repo locally	git clone https://github.com/...
git branch	List or create branches	git branch feature-login
git checkout <branch>	Switch to a branch	git checkout feature-login
git merge <branch>	Merge branch into current	git merge feature-login

9.3 Uploading a Project to GitHub

Follow these steps to upload your first project:

Step 1: Create a new repository on github.com (click + → New repository).

Step 2: Copy the repository URL (e.g., https://github.com/yourusername/my-project.git).

Step 3: Open your terminal in your project folder.

Step 4: Run: git init

Step 5: Run: git add .

Step 6: Run: git commit -m "Initial commit"

Step 7: Run: git branch -M main

Step 8: Run: git remote add origin

Step 9: Run: git push -u origin main

Step 10: Refresh GitHub — your files are now live!

Chapter 9 — Exercises & Self-Assessment

Part A: Multiple Choice Questions

1. What does git commit do?

- A) Uploads to GitHub
- B) Saves staged changes to local history
- C) Creates a new branch
- D) Deletes old files

2. What is a branch in Git?

- A) A copy of the entire project on GitHub
- B) An independent line of development
- C) A commit message
- D) A folder in the repository

3. What command stages ALL changes for commit?

- A) git stage .
- B) git push .
- C) git add .
- D) git save .

4. What is a pull request on GitHub?

- A) Downloading code
- B) A request to merge one branch into another
- C) Pulling from database
- D) Requesting collaborator access

5. What file describes a project on GitHub?

- A) NOTES.txt
- B) package.json
- C) README.md
- D) .gitignore

Part B: Short Answer Questions

1. What is the difference between git add and git commit?

2. Why should you use branches instead of committing directly to main?

3. What is .gitignore and why do you need it?

4. Explain the difference between git pull and git clone.

5. What information should a good README.md file contain?

Part C: Practical Exercises

Exercise 1: Create a new project folder, initialise Git, add 3 files, make 2 commits, create a branch, make a change, and merge it back to main.

Exercise 2: Upload your HTML/CSS project from Chapter 2 or 3 to a new GitHub repository.

Part D: Real-World Scenario

You and two classmates are building a project together. Describe your Git workflow: how would you set up the repository, how would each person work on different features without conflicts, how would you review each other's code, and how would you handle merge conflicts?

Answer Key

• MCQ: 1-B, 2-B, 3-C, 4-B, 5-C

• Short 1: git add stages (selects) changes; git commit saves those staged changes to history.

• Short 2: Branches isolate changes; you can experiment without breaking working code.

• Short 3: .gitignore lists files Git should not track (node_modules, .env, build files).

• Short 4: git clone copies an entire repo for the first time; git pull updates an existing local copy.

• Short 5: Project name, description, setup instructions, usage, technologies used, license.

CHAPTER 10

Deployment

Taking Your App from Localhost to the World

10.1 What is Deployment?

Deployment is the process of making your application available on the internet so anyone can use it. When you build on your laptop at localhost:3000, only you can see it. Deployment puts it on a server that's always on and accessible worldwide.

10.2 Deploying Frontend to Vercel

Step 1: Push your project to GitHub.

Step 2: Go to vercel.com and sign up with GitHub.

Step 3: Click 'New Project' and select your GitHub repository.

Step 4: Vercel auto-detects your framework (React, Next.js, etc.).

Step 5: Add environment variables if needed.

Step 6: Click Deploy — Vercel builds and hosts your app.

Step 7: Your app is live at yourusername-projectname.vercel.app

10.3 Deploying Backend to Render

```
# Ensure your server listens on process.env.PORT
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server on port ${PORT}`));

# Add start script to package.json
# "scripts": { "start": "node index.js" }
```

10.4 Common Deployment Errors

Error	Cause	Fix
Build failed	Syntax error or missing dependency	Check npm build output, fix errors
Port already in use	Server port hardcoded	Use process.env.PORT
API not found (404)	Wrong URL in frontend fetch	Check production API URL
CORS error	Backend blocks frontend origin	Add cors() middleware with correct origin
Environment var missing	Not set in deployment settings	Add in Vercel/Render dashboard

Chapter 10 — Exercises & Self-Assessment

Part A: Multiple Choice Questions

1. What is the difference between localhost and a live domain?

- A) localhost is faster
- B) localhost only works on your machine
- C) They are the same
- D) Live domains require payment always

2. Where do you set environment variables for a Vercel deployment?

- A) In .env file uploaded to GitHub
- B) In Vercel project settings dashboard
- C) In package.json
- D) In the HTML file

3. What Render setting change is needed for an Express app?

- A) Set node version
- B) Add MongoDB
- C) Listen on process.env.PORT
- D) Use HTTPS manually

4. What does a CORS error mean?

- A) Missing database
- B) Wrong password
- C) Server blocking requests from a different origin
- D) Syntax error

5. Which of these is a free frontend deployment platform?

- A) AWS EC2
- B) Vercel
- C) Google Cloud VM
- D) DigitalOcean Droplet

Part B: Short Answer Questions

1. What is the difference between frontend deployment and backend deployment?

2. Why should you never commit .env files to GitHub?

3. What is a CDN and how does Vercel use it?

4. What does it mean when your deployment says 'build failed'?

5. List 3 steps to test a newly deployed web application.

Part C: Practical Exercises

Exercise 1: Deploy your React Task Manager app from Chapter 5 to Vercel. Share the live URL.

Exercise 2: Deploy your Express CRUD API from Chapter 7 to Render. Test all endpoints with a tool like Postman or the browser.

Part D: Real-World Scenario

Your full-stack app (React frontend + Node.js backend + PostgreSQL database) is ready to deploy. Describe the exact steps: where will each part live, how will the frontend know the backend URL, and how will the backend connect to the production database?

Answer Key

• MCQ: 1-B, 2-B, 3-C, 4-C, 5-B

• Short 1: Frontend (HTML/CSS/JS files) → Vercel/Netlify; Backend (server code) → Render/Railway/Heroku.

• Short 2: .env contains secret keys and passwords; if pushed to GitHub, anyone can steal them.

• Short 3: CDN = Content Delivery Network; serves files from servers closest to each user for speed.

• Short 4: The code has errors — check the build logs for the specific error message.

• Short 5: Open the live URL in browser; test each page/feature; check browser console for errors.

CHAPTER P

FSD Practical Projects

Apply Everything You've Learned

Project 1: Student Portfolio Website

Build a personal portfolio to showcase your skills and projects.

Required Features

- Navigation bar with smooth scroll links
- Hero section with your name and tagline
- About Me section with skills list
- Projects section with 3+ project cards
- Contact form (name, email, message)
- Responsive design (mobile-friendly)

Tech Stack

HTML5, CSS3 (Flexbox/Grid), JavaScript (form validation)

Folder Structure

```
portfolio/
■■■ index.html
■■■ css/
■ ■■■ style.css
■■■ js/
■ ■■■ main.js
■■■ images/
■■■ profile.jpg
```

Project 2: Expense Tracker (React)

Track income and expenses with category filtering.

Features

- Add transaction (description, amount, type: income/expense, category)
- Display transaction list
- Show total balance, total income, total expenses
- Filter by category
- Delete transactions
- localStorage persistence

Core State

```
const [transactions, setTransactions] = useState([
{ id: 1, description: 'Salary', amount: 50000, type: 'income', category: 'Work' },
{ id: 2, description: 'Groceries', amount: 2500, type: 'expense', category: 'Food'
},
]);
```

Project 3: Full Stack Task Management System

A complete app with React frontend, Node.js/Express backend, and PostgreSQL database.

Endpoint	Method	Description
/api/tasks	GET	Get all tasks
/api/tasks	POST	Create new task
/api/tasks/:id	PUT	Update task
/api/tasks/:id	DELETE	Delete task
/api/auth/register	POST	Register user
/api/auth/login	POST	Login user

Project 4: Final Capstone Project

Build a complete full-stack application that demonstrates all your skills.

Choose One:

- E-Commerce Store: Product listing, cart, checkout, order management
- Social Platform: User profiles, posts, comments, likes, follow system
- Booking System: Services/slots, booking form, admin dashboard, confirmation emails

Evaluation Criteria

Criterion	Marks	What Evaluators Look For
Functionality	25	All features work as described
Code Quality	20	Clean, readable, commented code
UI/UX Design	20	Attractive, responsive interface
Database Design	15	Proper schema, relationships, queries
Deployment	10	Live and accessible URL
Documentation	10	README, code comments, API docs

Official Learning Resources

MDN Web Docs — Learn Web Development

https://developer.mozilla.org/en-US/docs/Learn_web_development

React Official Documentation

https://react.dev/learn

Next.js Documentation

https://nextjs.org/docs

GitHub Getting Started

https://docs.github.com/en/get-started

PostgreSQL Documentation

https://www.postgresql.org/docs/

Vercel Deployment Docs

https://vercel.com/docs

Glossary

API — Application Programming Interface. *Example: A set of rules for how programs communicate. REST API lets frontend talk to backend.*

Async/Await — Asynchronous JavaScript syntax. *Example: await fetch() waits for data without freezing the page.*

Backend — Server-side code and logic. *Example: Node.js handles login: checks password, returns token.*

Component — Reusable piece of UI in React. *Example: component used 10 times across the app.*

CSS — Cascading Style Sheets. *Example: color: purple; makes text purple.*

CRUD — Create, Read, Update, Delete. *Example: The four basic database/API operations.*

DOM — Document Object Model. *Example: JavaScript's way of accessing HTML elements on the page.*

Express.js — Node.js web framework. *Example: app.get('/users', ...) creates a GET endpoint.*

Frontend — User interface code. *Example: HTML, CSS, JavaScript in the browser.*

Git — Version control system. *Example: git commit -m 'Fix bug' saves a snapshot.*

GitHub — Cloud platform for Git repos. *Example: Share code: github.com/username/project*

Hook — React function for state/lifecycle. *Example: useState, useEffect are React hooks.*

HTML — HyperText Markup Language. *Example: Title creates a heading.*

HTTP — HyperText Transfer Protocol. *Example: GET /api/users fetches the users list.*

JSON — JavaScript Object Notation. *Example: {"name": "Priya", "age": 21}*

JSX — JavaScript XML (React syntax). *Example: Hello {name} in a React component.*

Next.js — React framework with SSR/routing. *Example: app/page.js → accessible at / URL.*

Node.js — JavaScript on the server. *Example: Run: node server.js to start a backend.*

ORM — Object Relational Mapper. *Example: Prisma lets you query DB with JS: await db.user.findMany()*

Props — Properties passed to React components. *Example: passes title as prop.*

React — JavaScript UI library. *Example: const [count, setCount] = useState(0);*

REST — Representational State Transfer. *Example: GET /api/books returns the book list.*

SQL — Structured Query Language. *Example: SELECT * FROM users WHERE age > 18;*

SSR — Server-Side Rendering. *Example: HTML rendered on server; good for SEO.*

State — Local component data in React. *Example: useState stores and updates form input values.*

Vercel — Frontend deployment platform. *Example: Push to GitHub → auto-deploy at yourapp.vercel.app*